



UCCD2044 Object-Oriented Programming Practices

Lecturer's name: Dr. Sayed Ahmad Zikri

GROUP 33

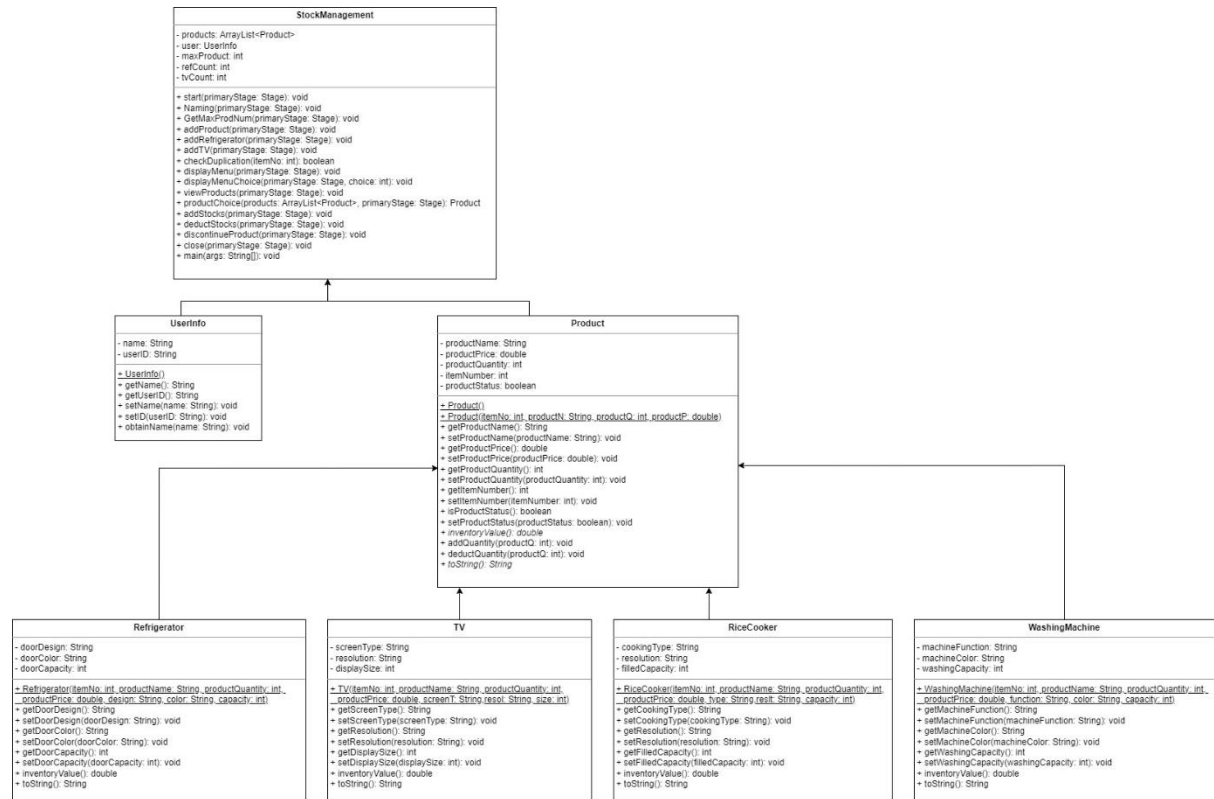
Stock Management System

No.	Name	Student ID	Course	Tutorial Group
1	Liew Ren Yi	2300434	CN	P1
2	Foo Kar Yeng	2300422	CN	P1
3	Tan Yi Fei	2200842	CN	P1
4	Wong Pei Kei	2207466	CN	P1

Task Distributions

Name & Student ID	Task Distribution
Liew Ren Yi 2300434	1. Main page (start, main) 2. Enter username - space/non-space, error handle (naming) 3. User checkout (close)
Tan Yi Fei 2200842	1. Choose product type (GetMaxProdNum) 2. Add product - error handle, count product (addRefrigerator, addTV, addRiceCooker, addWashingMachine) 3. Check duplication
Foo Kar Yeng 2300422	1. Menu - error handle (displayMenu, displayMenuChoice) 2. Display product list (viewProducts)
Wong Pei Kei 2207466	1. Choose product to update (productChoice) 2. Add stocks 3. Deduct stocks 4. Discontinue product

UML Diagram



Source Code

StockMangement.java

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;
import javax.swing.JOptionPane;
import javafx.application.Application;
import javafx.application.Platform;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Alert;
import javafx.scene.control.Button;
import javafx.scene.control.ChoiceDialog;
import javafx.scene.control.Label;
import javafx.scene.control.ScrollPane;
import javafx.scene.control.TextField;
import javafx.scene.control.TextFormatter;
import javafx.scene.control.TextInputDialog;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.layout.BorderPane;
import javafx.scene.layout.GridPane;
import javafx.scene.layout.HBox;
import javafx.scene.layout.VBox;
import javafx.scene.paint.Color;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
import javafx.scene.text.Text;
import javafx.stage.Stage;

public class StockManagement extends Application {

    static ArrayList<Product> products = new ArrayList<Product>();
    static UserInfo user = new UserInfo();
    static int maxProduct, refCount = 1, tvCount = 1, rcCount = 1, wmCount = 1;

    // Pane 1 - Welcome Message
    @Override
    public void start(Stage primaryStage) {

        ImageView logo = new ImageView("logo.png");
        logo.setFitWidth(100);
        logo.setFitHeight(100);
```

```

Label welcome = new Label("Welcome to the SMS!");
welcome.setTextFill(Color.DARKBLUE);
welcome.setFont(Font.font("Bauhaus 93", FontWeight.NORMAL, 30));
welcome.setAlignment(Pos.CENTER);

Label member = new Label("Group Members:" + "\n1. Foo Kar Yeng" + "\n2.
Liew Ren Yi" + "\n3. Tan Yi Fei"
                    + "\n4. Wong Pei Kei");
member.setFont(Font.font("Berlin Sans FB", 15));
member.setTextFill(Color.SLATEBLUE);

Button next = new Button("Create User");
next.setAlignment(Pos.CENTER);
next.setPadding(new Insets(5));

next.setOnAction(e -> Naming(primaryStage));

VBox vbox = new VBox();
vbox.getChildren().addAll(logo, welcome, member, next);
vbox.setAlignment(Pos.CENTER);
vbox.setPrefSize(300, 430);
vbox.setSpacing(10);

LocalDateTime currentDateTime = LocalDateTime.now();
DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd
HH:mm:ss");
String formattedDateTime = currentDateTime.format(formatter);

Label dateTime = new Label("Current Date and Time: " +
formattedDateTime);
dateTime.setFont(Font.font("Berlin Sans FB", 15));
dateTime.setTextFill(Color.SLATEBLUE);

HBox time = new HBox();
time.getChildren().add(dateTime);
time.setAlignment(Pos.BOTTOM_LEFT);

BorderPane root = new BorderPane();
root.setCenter(vbox);
root.setBottom(time);
root.setPadding(new Insets(15));
root.setStyle("-fx-background-color: ALICEBLUE");

Scene scene = new Scene(root, 550, 450);
primaryStage.getIcons().add(new Image("logo.png"));
primaryStage.setScene(scene);
primaryStage.setTitle("Stock Management System");
primaryStage.show();
}

```

```

// Pane 2 - Getting name from user
public void Naming(Stage primaryStage) {

    Label nameRequired = new Label("Please enter your first name and
surname:\n\n");
    nameRequired.setFont(Font.font("Berlin Sans FB", 18));
    nameRequired.setTextFill(Color.DEEPSKYBLUE);

    TextField name = new TextField();
    name.setPrefColumnCount(50);
    VBox field = new VBox(nameRequired, name);

    // Reject if name contains number or symbol
    TextFormatter<String> textFormatter = new TextFormatter<>(e -> {
        if (e.getText().matches("[0-9\\p{Punct}]")) {
            return null;
        }
        return e;
    });
    name.setTextFormatter(textFormatter);

    Button next = new Button("Next");
    VBox button = new VBox();
    button.getChildren().add(next);
    button.setAlignment(Pos.TOP_RIGHT);

    // Set action on "Next" button
    next.setOnAction(e -> {
        user.obtainName(name.getText());
        GetMaxProdNum(primaryStage);
    });

    GridPane pane2 = new GridPane();
    pane2.add(field, 0, 0);
    pane2.add(button, 0, 1);
    pane2.setPadding(new Insets(10));
    pane2.setVgap(5);
    pane2.setStyle("-fx-background-color: ALICEBLUE");

    Scene scene = new Scene(pane2, 450, 200);
    primaryStage.setScene(scene);
    primaryStage.setTitle("Stock Management System");
    primaryStage.show();

}

// Get maximum number of product
public static void GetMaxProdNum(Stage primaryStage) {

    TextField text = new TextField();

```

```

Label invalidMessage = new Label();
Button next = new Button("Next");
VBox button = new VBox(next);
button.setAlignment(Pos.TOP_RIGHT);

next.setOnAction(e -> {

    boolean valid = true;

    try {
        maxProduct = Integer.parseInt(text.getText());
        if (maxProduct < 0) {
            invalidMessage.setText("Invalid input, value must be a
positive value.");
            invalidMessage.setStyle("-fx-text-fill: red; -fx-font-style:
italic;");
            valid = false;
        }
    } catch (NumberFormatException ex) {
        invalidMessage.setText("Invalid input, Please enter a valid
number.");
        invalidMessage.setStyle("-fx-text-fill: red; -fx-font-style:
italic;");
        valid = false;
    }
    if (valid && maxProduct >= 0)
        addProduct(primaryStage);
});

Label required = new Label("Hi " + user.getName() + "!\n" + "Please enter
how many products you wish to add: ");
required.setFont(Font.font("Berlin Sans FB", 18));
required.setTextFill(Color.DEEPSKYBLUE);

GridPane pane = new GridPane();
pane.add(required, 0, 1);
pane.add(text, 0, 2);
pane.add(invalidMessage, 0, 3);
pane.add(button, 0, 4);

pane.setAlignment(Pos.CENTER);
pane.setPadding(new Insets(10, 15, 10, 15));
pane.setHgap(5);
pane.setVgap(5);
pane.setStyle("-fx-background-color: ALICEBLUE");

Scene scene = new Scene(pane, 450, 200);
primaryStage.setScene(scene);
primaryStage.setTitle("Stock Management System");
}

```

```

// choose product type to add into array list
public static void addProduct(Stage primaryStage) {

    if (maxProduct > 0) {
        Label required = new Label("Please choose a product type:");
        required.setFont(Font.font("Berlin Sans FB", 17));
        required.setTextFill(Color.DARKBLUE);

        Button addRef = new Button("Add refrigerator");
        addRef.setFont(Font.font("Berlin Sans FB", 17));
        addRef.setTextFill(Color.DARKBLUE);
        Button addTV = new Button("Add TV");
        addTV.setFont(Font.font("Berlin Sans FB", 17));
        addTV.setTextFill(Color.DARKBLUE);
        Button addRC = new Button("Add Rice Cooker");
        addRC.setFont(Font.font("Berlin Sans FB", 17));
        addRC.setTextFill(Color.DARKBLUE);
        Button addWM = new Button("Add Washing Machine");
        addWM.setFont(Font.font("Berlin Sans FB", 17));
        addWM.setTextFill(Color.DARKBLUE);

        addRef.setPrefSize(200, 25);
        addTV.setPrefSize(200, 25);
        addRC.setPrefSize(200, 25);
        addWM.setPrefSize(200, 25);

        addRef.setOnAction(e -> {
            addRefrigerator(primaryStage);
        });
        addTV.setOnAction(e -> {
            addTV(primaryStage);
        });
        addRC.setOnAction(e -> {
            addRiceCooker(primaryStage);
        });
        addWM.setOnAction(e -> {
            addWashingMachine(primaryStage);
        });

        GridPane pane2 = new GridPane();
        pane2.setPadding(new Insets(10, 15, 10, 15));
        pane2.setHgap(8);
        pane2.setVgap(8);
        pane2.setAlignment(Pos.CENTER);
        pane2.add(required, 0, 0);
        pane2.add(addRef, 0, 1);
        pane2.add(addTV, 0, 2);
        pane2.add(addRC, 0, 3);
        pane2.add(addWM, 0, 4);
    }
}

```



```

pane2.setStyle("-fx-background-color: ALICEBLUE");

Scene scene1 = new Scene(pane2, 300, 250);
primaryStage.setScene(scene1);
primaryStage.setTitle("Stock Management System");
primaryStage.show();
} else
    displayMenu(primaryStage);
}

```

```

public static void addRefrigerator(Stage primaryStage) {

```

```

    TextField nameField = new TextField();
    TextField designField = new TextField();
    TextField colorField = new TextField();
    TextField capacityField = new TextField();
    TextField quantityField = new TextField();
    TextField priceField = new TextField();
    TextField itemNoField = new TextField();
    Label invalidMessage = new Label();

```

```

    Button addButton = new Button("Add Refrigerator");
    addButton.setTextFill(Color.DARKBLUE);
    addButton.setFont(Font.font("Berlin Sans FB", 17));
    HBox button = new HBox(addButton);
    button.setAlignment(Pos.CENTER);

```

```

    // Set action on "add" button
    addButton.setOnAction(event -> {

```

```

        boolean valid = true;
        String name = null, design = null, color = null;
        int capacity = 0, quantity = 0, itemNo = 0;
        double price = 0;

```

```

        try {
            name = nameField.getText();
            design = designField.getText();
            color = colorField.getText();
            capacity = Integer.parseInt(capacityField.getText());
            quantity = Integer.parseInt(quantityField.getText());
            price = Double.parseDouble(priceField.getText());
            itemNo = Integer.parseInt(itemNoField.getText());
        }

```

```

        catch (NumberFormatException ex) {
            invalidMessage.setText("Invalid input, please insert proper

```

```

value.");

```

```

        invalidMessage.setStyle("-fx-text-fill: red; -fx-font-style:
italic;");
        valid = false;
    }

    // Check product duplication
    if (checkDuplication(itemNo)) {
        invalidMessage.setText("Duplicate item number. Please re-
enter item number.");
        invalidMessage.setStyle("-fx-text-fill: red; -fx-font-style:
italic;");
        valid = false;
    }

    if (valid) {
        Product newRef = new Refrigerator(itemNo, name, quantity,
price, design, color, capacity);
        products.add(newRef);

        refCount++;
        // Add more than one products
        maxProduct--;
        addProduct(primaryStage);
    }
});

```

```

GridPane gp = new GridPane();
gp.setMinSize(400, 200);
gp.setPadding(new Insets(10, 10, 10, 10));
gp.setVgap(10);
gp.setHgap(15);
gp.setAlignment(Pos.CENTER);

gp.add(new Label("Name:"), 0, 0);
gp.add(nameField, 1, 0);
gp.add(new Label("Door Design:"), 0, 1);
gp.add(designField, 1, 1);
gp.add(new Label("Door Color:"), 0, 2);
gp.add(colorField, 1, 2);
gp.add(new Label("Door Capacity:"), 0, 3);
gp.add(capacityField, 1, 3);
gp.add(new Label("Quantity Available in Stock:"), 0, 4);
gp.add(quantityField, 1, 4);
gp.add(new Label("Price:"), 0, 5);
gp.add(priceField, 1, 5);
gp.add(new Label("Item Number:"), 0, 6);
gp.add(itemNoField, 1, 6);

```

```

VBox pane = new VBox(gp, invalidMessage, button);
pane.setAlignment(Pos.CENTER);

```

```

pane.setStyle("-fx-background-color: ALICEBLUE");

Scene scene = new Scene(pane, 420, 370);
primaryStage.setScene(scene);
primaryStage.setTitle("Add Refrigerator " + refCount);
primaryStage.show();

}

public static void addTV(Stage primaryStage) {

    TextField nameField = new TextField();
    TextField screenTypeField = new TextField();
    TextField resolutionField = new TextField();
    TextField displaySizeField = new TextField();
    TextField quantityField = new TextField();
    TextField priceField = new TextField();
    TextField itemNoField = new TextField();
    Label invalidMessage = new Label();

    Button addButton = new Button("Add TV");
    addButton.setTextFill(Color.DARKBLUE);
    addButton.setFont(Font.font("Berlin Sans FB", 17));
    HBox button = new HBox(addButton);
    button.setAlignment(Pos.CENTER);

    // Set action on "add" button
    addButton.setOnAction(event -> {

        boolean valid = true;
        String name = null, screenType = null, resolution = null;
        int displaySize = 0, quantity = 0, itemNo = 0;
        double price = 0;

        try {
            name = nameField.getText();
            screenType = screenTypeField.getText();
            resolution = resolutionField.getText();
            displaySize = Integer.parseInt(displaySizeField.getText());
            quantity = Integer.parseInt(quantityField.getText());
            price = Double.parseDouble(priceField.getText());
            itemNo = Integer.parseInt(itemNoField.getText());

        }

        catch (NumberFormatException ex) {
            invalidMessage.setText("Invalid input, please insert proper
value.");
            invalidMessage.setStyle("-fx-text-fill: red; -fx-font-style:
italic;");

```

```

        valid = false;
    }

    // Check product duplication
    if (checkDuplication(itemNo)) {
        invalidMessage.setText("Duplicate item number. Please re-
enter item number.");
        invalidMessage.setStyle("-fx-text-fill: red; -fx-font-style:
italic;");
        valid = false;
    }

    else if (valid) {

        Product newTV = new TV(itemNo, name, quantity, price,
screenType, resolution, displaySize);
        products.add(newTV);

        tvCount++;
        maxProduct--;
        addProduct(primaryStage);
    }
});

GridPane gp = new GridPane();
gp.setMinSize(400, 200);
gp.setPadding(new Insets(10, 10, 10, 10));
gp.setVgap(10);
gp.setHgap(15);
gp.setAlignment(Pos.CENTER);

gp.add(new Label("Name:"), 0, 0);
gp.add(nameField, 1, 0);
gp.add(new Label("Screen type:"), 0, 1);
gp.add(screenTypeField, 1, 1);
gp.add(new Label("Resolution:"), 0, 2);
gp.add(resolutionField, 1, 2);
gp.add(new Label("Display Size:"), 0, 3);
gp.add(displaySizeField, 1, 3);
gp.add(new Label("Quantity Available in Stock:"), 0, 4);
gp.add(quantityField, 1, 4);
gp.add(new Label("Price:"), 0, 5);
gp.add(priceField, 1, 5);
gp.add(new Label("Item Number:"), 0, 6);
gp.add(itemNoField, 1, 6);

VBox pane = new VBox(gp, invalidMessage, button);
pane.setAlignment(Pos.CENTER);
pane.setStyle("-fx-background-color: ALICEBLUE");

```

```

        Scene scene = new Scene(pane, 420, 370);
        primaryStage.setScene(scene);
        primaryStage.setTitle("Add TV " + tvCount);
        primaryStage.show();
    }

    public static void addRiceCooker(Stage primaryStage) {

        TextField nameField = new TextField();
        TextField typeField = new TextField();
        TextField resolutionField = new TextField();
        TextField capacityField = new TextField();
        TextField quantityField = new TextField();
        TextField priceField = new TextField();
        TextField itemNoField = new TextField();
        Label invalidMessage = new Label();

        Button addButton = new Button("Add Rice Cooker");
        addButton.setTextFill(Color.DARKBLUE);
        addButton.setFont(Font.font("Berlin Sans FB", 17));
        HBox button = new HBox(addButton);
        button.setAlignment(Pos.CENTER);

        // Set action on "add" button
        addButton.setOnAction(event -> {

            boolean valid = true;
            String name = null, type = null, reslt = null;
            int capacity = 0, quantity = 0, itemNo = 0;
            double price = 0;

            try {
                name = nameField.getText();
                type = typeField.getText();
                reslt = resolutionField.getText();
                capacity = Integer.parseInt(capacityField.getText());
                quantity = Integer.parseInt(quantityField.getText());
                price = Double.parseDouble(priceField.getText());
                itemNo = Integer.parseInt(itemNoField.getText());
            }

            catch (NumberFormatException ex) {
                invalidMessage.setText("Invalid input, please insert proper
value.");
                invalidMessage.setStyle("-fx-text-fill: red; -fx-font-style:
italic;");
                valid = false;
            }

```

```

        // Check product duplication
        if (checkDuplication(itemNo)) {
            invalidMessage.setText("Duplicate item number. Please re-
enter item number.");
            invalidMessage.setStyle("-fx-text-fill: red; -fx-font-style:
italic;");
            valid = false;
        }

        if (valid) {
            Product newRC = new RiceCooker(itemNo, name, quantity,
price, type, reslt, capacity);
            products.add(newRC);

            rcCount++;
            // Add more than one products
            maxProduct--;
            addProduct(primaryStage);
        }
    });

```

```

GridPane gp = new GridPane();
gp.setMinSize(400, 200);
gp.setPadding(new Insets(10, 10, 10, 10));
gp.setVgap(10);
gp.setHgap(15);
gp.setAlignment(Pos.CENTER);

gp.add(new Label("Name:"), 0, 0);
gp.add(nameField, 1, 0);
gp.add(new Label("Cooker Type:"), 0, 1);
gp.add(typeField, 1, 1);
gp.add(new Label("Resolution:"), 0, 2);
gp.add(resolutionField, 1, 2);
gp.add(new Label("Volume:"), 0, 3);
gp.add(capacityField, 1, 3);
gp.add(new Label("Quantity Available in Stock:"), 0, 4);
gp.add(quantityField, 1, 4);
gp.add(new Label("Price:"), 0, 5);
gp.add(priceField, 1, 5);
gp.add(new Label("Item Number:"), 0, 6);
gp.add(itemNoField, 1, 6);

```

```

VBox pane = new VBox(gp, invalidMessage, button);
pane.setAlignment(Pos.CENTER);
pane.setStyle("-fx-background-color: ALICEBLUE");

```

```

Scene scene = new Scene(pane, 420, 370);
primaryStage.setScene(scene);
primaryStage.setTitle("Add Rice Cooker " + rcCount);

```

```

        primaryStage.show();
    }

    public static void addWashingMachine(Stage primaryStage) {

        TextField nameField = new TextField();
        TextField functionField = new TextField();
        TextField colorField = new TextField();
        TextField capacityField = new TextField();
        TextField quantityField = new TextField();
        TextField priceField = new TextField();
        TextField itemNoField = new TextField();
        Label invalidMessage = new Label();

        Button addButton = new Button("Add Washing Machine");
        addButton.setTextFill(Color.DARKBLUE);
        addButton.setFont(Font.font("Berlin Sans FB", 17));
        HBox button = new HBox(addButton);
        button.setAlignment(Pos.CENTER);

        // Set action on "add" button
        addButton.setOnAction(event -> {

            boolean valid = true;
            String name = null, function = null, color = null;
            int capacity = 0, quantity = 0, itemNo = 0;
            double price = 0;

            try {
                name = nameField.getText();
                function = functionField.getText();
                color = colorField.getText();
                capacity = Integer.parseInt(capacityField.getText());
                quantity = Integer.parseInt(quantityField.getText());
                price = Double.parseDouble(priceField.getText());
                itemNo = Integer.parseInt(itemNoField.getText());
            }

            catch (NumberFormatException ex) {
                invalidMessage.setText("Invalid input, please insert proper
value.");
                invalidMessage.setStyle("-fx-text-fill: red; -fx-font-style:
italic;");
                valid = false;
            }

            // Check product duplication
            if (checkDuplication(itemNo)) {

```

```

        invalidMessage.setText("Duplicate item number. Please re-
enter item number.");
        invalidMessage.setStyle("-fx-text-fill: red; -fx-font-style:
italic;");
        valid = false;
    }

    if (valid) {
        Product newWM = new WashingMachine(itemNo, name,
quantity, price, function, color, capacity);
        products.add(newWM);

        wmCount++;
        // Add more than one products
        maxProduct--;
        addProduct(primaryStage);
    }
});

GridPane gp = new GridPane();
gp.setMinSize(400, 200);
gp.setPadding(new Insets(10, 10, 10, 10));
gp.setVgap(10);
gp.setHgap(15);
gp.setAlignment(Pos.CENTER);

gp.add(new Label("Name:"), 0, 0);
gp.add(nameField, 1, 0);
gp.add(new Label("Machine Function:"), 0, 1);
gp.add(functionField, 1, 1);
gp.add(new Label("Machine Color:"), 0, 2);
gp.add(colorField, 1, 2);
gp.add(new Label("Volume:"), 0, 3);
gp.add(capacityField, 1, 3);
gp.add(new Label("Quantity Available in Stock:"), 0, 4);
gp.add(quantityField, 1, 4);
gp.add(new Label("Price:"), 0, 5);
gp.add(priceField, 1, 5);
gp.add(new Label("Item Number:"), 0, 6);
gp.add(itemNoField, 1, 6);

VBox pane = new VBox(gp, invalidMessage, button);
pane.setAlignment(Pos.CENTER);
pane.setStyle("-fx-background-color: ALICEBLUE");

Scene scene = new Scene(pane, 420, 370);
primaryStage.setScene(scene);
primaryStage.setTitle("Add Washing Machine " + wmCount);
primaryStage.show();

```



```

    }

    // Each product itemNo must be unique (no duplication)
    public static boolean checkDuplication(int itemNo) {

        for (int i = 0; i < products.size(); i++) {
            if (products.get(i).getItemNumber() == itemNo) {
                return true;
            }
        }
        return false;
    }

    // Display menu
    public static void displayMenu(Stage primaryStage) {

        Text menu = new Text("MENU");
        menu.setFont(Font.font("Bauhaus 93", 30));
        menu.setFill(Color.DARKBLUE);
        VBox menuBox = new VBox(menu);
        menuBox.setAlignment(Pos.CENTER);

        VBox optionBox = new VBox();
        optionBox.getChildren().add(new Label("1. View Products"));
        optionBox.getChildren().add(new Label("2. Add Stock"));
        optionBox.getChildren().add(new Label("3. Deduct Stock"));
        optionBox.getChildren().add(new Label("4. Discontinue product"));
        optionBox.getChildren().add(new Label("0. Exit"));
        optionBox.setAlignment(Pos.CENTER);
        optionBox.setStyle("-fx-border-color: LIGHTBLUE; -fx-border-width: 5;");
        optionBox.setPadding(new Insets(10, 0, 0, 0));

        TextField input = new TextField("");
        input.setPrefColumnCount(1);
        GridPane inputPane = new GridPane();
        inputPane.add(new Label("Please enter an option:"), 0, 0);
        inputPane.add(input, 1, 0);
        inputPane.setHgap(5);
        inputPane.setAlignment(Pos.CENTER);

        Label invalidMessage = new Label();

        optionBox.getChildren().addAll(inputPane, invalidMessage);

        Button next = new Button("Next");
        Button back = new Button("Add Products");
        BorderPane buttonPane = new BorderPane();
        buttonPane.setLeft(back);
        buttonPane.setRight(next);
    }

```

```

buttonPane.setPadding(new Insets(5));

next.setOnAction(e -> {

    int menuChoice = 0;
    boolean valid = true;

    try {
        menuChoice = Integer.parseInt(input.getText());

        if (menuChoice < 0) {
            invalidMessage.setText("Invalid input, the value must
be positive non-zero value.");
            invalidMessage.setStyle("-fx-text-fill: red; -fx-font-style:
italic;");
            valid = false;
        }
        else if(menuChoice > 4) {
            invalidMessage.setText("Invalid input, value must be
within the range of 0 to 4.");
            invalidMessage.setStyle("-fx-text-fill: red; -fx-font-style:
italic;");
            valid = false;
        }
    } catch (NumberFormatException ex) {
        invalidMessage.setText("Invalid input, please insert an
integer.");
        invalidMessage.setStyle("-fx-text-fill: red; -fx-font-style:
italic;");
        valid = false;
    }
    if (valid)
        displayMenuChoice(primaryStage, menuChoice);

});

back.setOnAction(e -> {
    maxProduct = 0;
    GetMaxProdNum(primaryStage);
});

BorderPane pane = new BorderPane();
pane.setTop(menuBox);
pane.setCenter(optionBox);
pane.setBottom(buttonPane);
pane.setPadding(new Insets(10));
pane.setStyle("-fx-background-color: ALICEBLUE");

Scene scene = new Scene(pane, 450, 280);
primaryStage.setTitle("Menu");

```

```

        primaryStage.setScene(scene);
        primaryStage.show();
    }

```

// Perform menu option

```

public static void displayMenuChoice(Stage primaryStage, int choice) {

```

```

    switch (choice) {
    case 1:
        viewProducts(primaryStage);
        break;
    case 2:
        addStocks(primaryStage);
        break;
    case 3:
        deductStocks(primaryStage);
        break;
    case 4:
        discontinueProduct(primaryStage);
        break;
    case 0:
        close(primaryStage);
        break;
    }
}

```

// Display Products List

```

public static void viewProducts(Stage primaryStage) {

```

```

    StringBuilder list = new StringBuilder();

```

```

    String type;

```

```

    int refCount = 0, tvCount = 0, rcCount = 0, wmCount = 0;

```

```

    for (Product product : products) {
        // if (product != null) {

```

```

            if (product instanceof Refrigerator) {
                type = "Refrigerator";
                refCount++;
                list.append(type + " " + refCount + product.toString() + "\n");
            } else if (product instanceof TV) {
                type = "TV";
                tvCount++;
                list.append(type + " " + tvCount + product.toString() + "\n");
            } else if (product instanceof RiceCooker) {
                type = "Rice Cooker";
                rcCount++;
                list.append(type + " " + rcCount + product.toString() + "\n");
            } else if (product instanceof WashingMachine) {

```

```

        type = "Washing Machine";
        wmCount++;
        list.append(type + " " + wmCount + product.toString() + "\n");
    }
}
// }

Text productList = new Text(list.toString());

ScrollPane scroll = new ScrollPane(productList);
scroll.setPrefSize(390, 220);
scroll.setPadding(new Insets(10));

Button back = new Button("Back");
back.setOnAction(e -> displayMenu(primaryStage));

BorderPane pane = new BorderPane();
pane.setCenter(scroll);
pane.setBottom(back);
pane.setPadding(new Insets(10));
pane.setStyle("-fx-background-color: ALICEBLUE");

Scene scene = new Scene(pane, 450, 300);
primaryStage.setScene(scene);
primaryStage.setTitle("Display Products");
primaryStage.show();
}

// Choose which product to update
public static Product productChoice(ArrayList<Product> products, Stage
primaryStage) {

    List<String> productName = new ArrayList<>();

    // Check if product list is empty
    if (!products.isEmpty()) {
        for (Product product : products)
            productName.add(product.getProductName());

        ChoiceDialog<String> dialog = new
ChoiceDialog<>(productName.get(0), productName);
        dialog.setTitle("Product Selection");
        dialog.setHeaderText("Please Select a Product to Update");
        dialog.setContentText("Available Product:");

        // Show the dialog and wait for the user's response
        Optional<String> choice = dialog.showAndWait();

        if (choice.isPresent()) {
            for (Product product : products)

```

```

        if (product.getProductName() == choice.get())
            return product;
    }
    return null; // User canceled the dialog
} else {
    // Handle the case where the product list is null or empty
    Alert alert = new Alert(Alert.AlertType.ERROR);
    alert.setTitle("Error");
    alert.setHeaderText(null);
    alert.setContentText("No products available.");
    alert.showAndWait();
    return null;
}
}

// Add stock for the chosen product
public static void addStocks(Stage primaryStage) {

    Product chosenProduct = productChoice(products, primaryStage);
    int stockValue = 0;
    boolean valid = true;

    if (chosenProduct != null) {
        if (!chosenProduct.isProductStatus()) {
            // Handle the case where no product is selected
            Alert alert = new Alert(Alert.AlertType.WARNING);
            alert.setTitle("Invalid Selection");
            alert.setHeaderText(null);
            alert.setContentText("This product discontinued, not allow to
add stock.");

            alert.showAndWait();
            return;
        }
        TextInputDialog dialog = new TextInputDialog();
        dialog.setTitle("Add Stock");
        dialog.setHeaderText("Please Enter Quantity to Add");
        dialog.setContentText("Stock Values:");

        Optional<String> result = dialog.showAndWait();

        // Check if user entered text
        if (result.isPresent()) {

            try {
                stockValue = Integer.parseInt(result.get());

                if (stockValue < 0) {
                    JOptionPane.showMessageDialog(null, "Invalid
input, please enter a postive value!", "Alert",

```

```

JOptionPane.ERROR_MESSAGE);
        valid = false;
        addStocks(primaryStage);
    }
} catch (NumberFormatException ex) {
    JOptionPane.showMessageDialog(null, "Invalid input,
please enter an integer!", "Alert",
        JOptionPane.ERROR_MESSAGE);
    valid = false;
    addStocks(primaryStage);
}
if(valid) {
    chosenProduct.addQuantity(stockValue);
    Alert alert = new
Alert(Alert.AlertType.INFORMATION);
    alert.setTitle("Status Updated");
    alert.setHeaderText(null);
    alert.setContentText("The stock of " +
chosenProduct.getProductNames() + " has been added by " + stockValue + " ");
    alert.showAndWait();
}
} else {
    JOptionPane.showMessageDialog(null, "Field is left empty, re-
enter again!", "Alert",
        JOptionPane.ERROR_MESSAGE);
    addStocks(primaryStage);
}
}
else {
    // Handle the case where no product is selected
    Alert alert = new Alert(Alert.AlertType.WARNING);
    alert.setTitle("No Selection");
    alert.setHeaderText(null);
    alert.setContentText("No product was selected.");
    alert.showAndWait();
}
}

// update number of products after deduction
public static void deductStocks(Stage primaryStage) {

    Product chosenProduct = productChoice(products, primaryStage);
    int stockValue = 0;
    boolean valid = true;

    if (chosenProduct != null) {
        TextInputDialog dialog = new TextInputDialog();
        dialog.setTitle("Deduct Stock");
        dialog.setHeaderText("Please Enter Quantity to Deduct");
    }
}

```

```

        dialog.setContentText("Stock Values:");

        Optional<String> result = dialog.showAndWait();

        // Check if user entered text
        if (result.isPresent()) {

            try {
                stockValue = Integer.parseInt(result.get());

                if (stockValue < 0) {
                    JOptionPane.showMessageDialog(null, "Invalid
input, please enter a postive value!", "Alert",
JOptionPane.ERROR_MESSAGE);
                    valid = false;
                    deductStocks(primaryStage);
                }
                if (stockValue >= chosenProduct.getProductQuantity())
{
                    JOptionPane.showMessageDialog(null, "Invalid
input, value must be within the product quantity!",
"Alert",
JOptionPane.ERROR_MESSAGE);
                    valid = false;
                    deductStocks(primaryStage);
                }
            } catch (NumberFormatException ex) {
                JOptionPane.showMessageDialog(null, "Invalid input,
please enter an integer!", "Status",
JOptionPane.ERROR_MESSAGE);
                valid = false;
                deductStocks(primaryStage);
            }
            if(valid) {
                chosenProduct.deductQuantity(stockValue);
                Alert alert = new
Alert(Alert.AlertType.INFORMATION);
                alert.setTitle("Status Updated");
                alert.setHeaderText(null);
                alert.setContentText("The stock of " +
chosenProduct.getProductName() + " has been deducted by " + stockValue + " ");
                alert.showAndWait();
            }

        } else {
            JOptionPane.showMessageDialog(null, "Field is left empty,
please re-enter again!", "Alert",
JOptionPane.ERROR_MESSAGE);
            deductStocks(primaryStage);
        }
    }
}

```

```

    }
}
else {
    // Handle the case where no product is selected
    Alert alert = new Alert(Alert.AlertType.WARNING);
    alert.setTitle("No Selection");
    alert.setHeaderText(null);
    alert.setContentText("No product was selected.");
    alert.showAndWait();
}
}

// set the chosen product to be discontinued
public static void discontinueProduct(Stage primaryStage) {

    Product chosenProduct = productChoice(products, primaryStage);
    if (chosenProduct != null) {
        chosenProduct.setProductStatus(false);
        Alert alert = new Alert(Alert.AlertType.INFORMATION);
        alert.setTitle("Status Updated");
        alert.setHeaderText(null);
        alert.setContentText("The status of " +
chosenProduct.getProductName() + " has been set to false.");
        alert.showAndWait();
    } else {
        // Handle the case where no product is selected
        Alert alert = new Alert(Alert.AlertType.WARNING);
        alert.setTitle("No Selection");
        alert.setHeaderText(null);
        alert.setContentText("No product was selected.");
        alert.showAndWait();
    }
}

// exit scene
public static void close(Stage primaryStage) {
    Text text1 = new Text("User ID : " + user.getUserID());
    Text text2 = new Text("User Name : " + user.getName());

    GridPane exit = new GridPane();
    Button ok = new Button("ok");

    ok.setOnAction(e -> Platform.exit());

    exit.setPadding(new Insets(10));
    exit.setAlignment(Pos.CENTER);
    exit.add(text1, 0, 0);
    exit.add(text2, 0, 1);
    exit.add(ok, 0, 2);
}

```



```
        Scene scene = new Scene(exit, 300, 100);
        primaryStage.setTitle("Check out");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    // main function
    public static void main(String[] args) {
        launch(args);
    }
}
```

Product.java

```
public abstract class Product {  
  
    private String productName;  
    private double productPrice;  
    private int productQuantity, itemNumber;  
    private boolean productStatus= true;  
  
    public Product() {  
        productStatus = true;  
    }  
  
    public Product(int itemNo, String productN, int productQ, double productP) {  
        itemNumber = itemNo;  
        productName = productN;  
        productQuantity = productQ;  
        productPrice = productP;  
    }  
  
    public String getProductName() {  
        return productName;  
    }  
  
    public void setProductName(String productName) {  
        this.productName = productName;  
    }  
  
    public double getProductPrice() {  
        return productPrice;  
    }  
  
    public void setProductPrice(double productPrice) {  
        this.productPrice = productPrice;  
    }  
  
    public int getProductQuantity() {  
        return productQuantity;  
    }  
  
    public void setProductQuantity(int productQuantity) {  
        this.productQuantity = productQuantity;  
    }  
  
    public int getItemNumber() {  
        return itemNumber;  
    }  
  
    public void setItemNumber(int itemNumber) {  
        this.itemNumber = itemNumber;  
    }  
}
```

```

    }

    public boolean isProductStatus() {
        return productStatus;
    }

    public void setProductStatus(boolean productStatus) {
        this.productStatus = productStatus;
    }

    public double inventoryValue() {
        return productPrice * productQuantity;
    }

    // Do not allow the user to add stock to a discontinued product
    public void addQuantity(int productQ) {
        if(productStatus)
            productQuantity += productQ;
        else
            System.out.println("Not allow to add stock to a discontinued
product.");
    }

    public void deductQuantity(int productQ) {
        productQuantity -= productQ;
    }

    @Override
    public String toString() {
        return "\nItem number\t: " + itemNumber + "\n" +
            "Product name\t: " + productName + "\n" +
            "Quantity available: " + productQuantity + "\n" +
            "Price (RM)\t: " + productPrice + "\n" +
            "Inventory value (RM):" + inventoryValue() + "\n" +
            "Product status: " + productStatus + "\n";
    }
}

```

Refrigerator.java

```
public class Refrigerator extends Product{

    private String doorDesign;
    private String doorColor;
    private int doorCapacity;

    public Refrigerator(int itemNo, String productName, int productQuantity, double
productPrice, String design, String color, int capacity) {
        super(itemNo, productName, productQuantity, productPrice);
        doorDesign = design;
        doorColor = color;
        doorCapacity = capacity;
    }

    public String getDoorDesign() {
        return doorDesign;
    }

    public void setDoorDesign(String doorDesign) {
        this.doorDesign = doorDesign;
    }

    public String getDoorColor() {
        return doorColor;
    }

    public void setDoorColor(String doorColor) {
        this.doorColor = doorColor;
    }

    public int getDoorCapacity() {
        return doorCapacity;
    }

    public void setDoorCapacity(int doorCapacity) {
        this.doorCapacity = doorCapacity;
    }

    public double inventoryValue() {
        return getProductPrice() * getProductQuantity();
    }

    @Override
    public String toString() {
        return "\nItem number\t\t: " + getItemNumber() + "\n" +
            "Product name\t\t: " + getProductName() + "\n" +
            "Door design\t\t: " + doorDesign + "\n" +
            "Color\t\t\t: " + doorColor + "\n" +
```

```
        "Capacity (in Litres)\t: " + doorCapacity + "\n" +  
        "Quantity available\t: " + getProductQuantity() + "\n" +  
        "Price (RM)\t\t: " + getProductPrice() + "\n" +  
        "Inventory value (RM): " + inventoryValue() + "\n" +  
        "Product status\t\t: " + isProductStatus() + "\n";  
    }  
}
```

RiceCooker.java

```
public class RiceCooker extends Product {
    private String cookingType;
    private String resolution;
    private int filledCapacity;

    public RiceCooker(int itemNo, String productName, int productQuantity, double
productPrice, String type,
        String reslt, int capacity) {
        super(itemNo, productName, productQuantity, productPrice);
        cookingType = type;
        resolution = reslt;
        filledCapacity = capacity;
    }

    public String getCookingType() {
        return cookingType;
    }

    public void setCookingType(String cookingType) {
        this.cookingType = cookingType;
    }

    public String getResolution() {
        return resolution;
    }

    public void setResolution(String resolution) {
        this.resolution = resolution;
    }

    public int getFilledCapacity() {
        return filledCapacity;
    }

    public void setFilledCapacity(int filledCapacity) {
        this.filledCapacity = filledCapacity;
    }

    public double inventoryValue() {
        return getProductPrice() * getProductQuantity();
    }

    @Override
    public String toString() {
        return "\nItem number\t\t: " + getItemNumber() + "\n" + "Product name\t\t: "
+ getProductName() + "\n"
        + "Cooker Type\t\t: " + cookingType + "\n" + "Resolution\t\t: "
+ resolution + "\n"
    }
}
```

```
        + "Volume (in litre)\t: " + filledCapacity + "\n" + "Quantity
available\t: " + getProductQuantity()
        + "\n" + "Price (RM)\t\t: " + getProductPrice() + "\n" +
"Inventory value (RM): " + inventoryValue()
        + "\n" + "Product status\t\t: " + isProductStatus() + "\n";
    }
}
```

TV.java

```
public class TV extends Product{

    private String screenType, resolution;
    private int displaySize;

    public TV(int itemNo, String productName, int productQuantity, double
productPrice, String screenT, String resol, int size) {
        super(itemNo, productName, productQuantity, productPrice);
        screenType = screenT;
        resolution = resol;
        displaySize = size;
    }

    public String getScreenType() {
        return screenType;
    }

    public void setScreenType(String screenType) {
        this.screenType = screenType;
    }

    public String getResolution() {
        return resolution;
    }

    public void setResolution(String resolution) {
        this.resolution = resolution;
    }

    public int getDisplaySize() {
        return displaySize;
    }

    public void setDisplaySize(int displaySize) {
        this.displaySize = displaySize;
    }

    public double inventoryValue() {
        return getProductPrice() * getProductQuantity();
    }

    @Override
    public String toString() {
        return "\nItem number\t\t: " + getItemNumber() + "\n" +
            "Product name\t\t: " + getProductName() + "\n" +
            "Screen Type\t\t: " + screenType + "\n" +
            "Resolution\t\t: " + resolution + "\n" +
            "Display size\t\t: " + displaySize + "\n" +
```



```
        "Quantity available\t: " + getProductQuantity() + "\n" +  
        "Price (RM)\t\t: " + getProductPrice() + "\n" +  
        "Inventory value (RM): " + inventoryValue() + "\n" +  
        "Product status\t\t: " + isProductStatus() + "\n";  
    }  
  
}
```

WashingMachine.java

```
public class WashingMachine extends Product {

    private String machineFunction;
    private String machineColor;
    private int washingCapacity;

    public WashingMachine(int itemNo, String productName, int productQuantity,
double productPrice,
        String function, String color, int capacity) {
        super(itemNo, productName, productQuantity, productPrice);
        machineFunction = function;
        machineColor = color;
        washingCapacity = capacity;
    }

    public String getMachineFunction() {
        return machineFunction;
    }

    public void setMachineFunction(String machineFunction) {
        this.machineFunction = machineFunction;
    }

    public String getMachineColor() {
        return machineColor;
    }

    public void setMachineColor(String machineColor) {
        this.machineColor = machineColor;
    }

    public int getWashingCapacity() {
        return washingCapacity;
    }

    public void setWashingCapacity(int washingCapacity) {
        this.washingCapacity = washingCapacity;
    }

    public double inventoryValue() {
        return getProductPrice() * getProductQuantity();
    }

    @Override
    public String toString() {
        return "\nItem number\t\t: " + getItemNumber() + "\n" + "Product name\t\t: "
+ getProductName() + "\n"
```

```
        + "Function\t\t\t: " + machineFunction + "\n" + "Color\t\t\t: " +  
machineColor + "\n"  
        + "Volume (in kg)\t: " + washingCapacity + "\n" + "Quantity  
available\t: " + getProductQuantity()  
        + "\n" + "Price (RM)\t\t: " + getProductPrice() + "\n" +  
"Inventory value (RM): " + inventoryValue()  
        + "\n" + "Product status\t\t: " + isProductStatus() + "\n";  
    }  
}
```

UserInfo.java

```
public class UserInfo {

    private String name;
    private String userID;

    public UserInfo() {
        name = " ";
        userID = " ";
    }

    public String getName() {
        return name;
    }

    public String getUserID() {
        return userID;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setID(String userID) {
        this.userID = userID;
    }

    // Validate the name and assign userID
    public void obtainName(String name) {
        setName(name);

        if (name.contains(" ")) {
            int index = name.lastIndexOf(" ");
            userID = name.substring(0, 1).toUpperCase() +
                name.substring(index + 1, index + 2).toUpperCase() +
name.substring(index + 2);
        }
        else
            userID = "guest";
    }

}
```